



UNIVERSIDAD
Blas Pascal

PROYECTO FINAL 2026 - TC

Asignatura: Técnicas de Compilación

Fecha: 12/6/2026

Autor:

Tomás Calzada
Octavio Douglas
Ignacio Galvan

Profesor: Francisco Ameri

RESUMEN

El presente informe describe el diseño y la implementación de un compilador educativo para un subconjunto de C++. El sistema fue desarrollado en Java con ANTLR4 y organiza el procesamiento en análisis léxico, sintáctico y semántico, generación de código intermedio y optimización.

La solución reconoce variables, arreglos básicos, expresiones, estructuras de control y funciones. Además, administra ámbitos mediante una tabla de símbolos, genera instrucciones de tres direcciones y guarda el resultado antes y después de la optimización. La verificación final ejecutó 17 pruebas automatizadas sin fallos ni errores.

1. INTRODUCCIÓN

El objetivo del proyecto fue aplicar de forma práctica los conceptos centrales de Técnicas de Compilación. Debido a que C++ completo posee una gramática y una semántica demasiado amplias para el alcance del trabajo, se definió un lenguaje reducido que conserva construcciones suficientes para estudiar las distintas fases de compilación.

El compilador recibe un archivo .txt o .cpp y procesa cada etapa en orden. Si encuentra un error crítico, detiene el flujo y muestra la línea, la columna y una descripción. Cuando la entrada es válida, presenta los tokens, el árbol de parseo y la tabla de símbolos, y genera archivos con el código intermedio y optimizado.

1.1 Objetivo general

Diseñar un compilador modular capaz de reconocer, validar y transformar programas escritos en un subconjunto de C++, mostrando de manera clara las fases internas y sus resultados.

1.2 Objetivos específicos

Gramática: definir tokens, sentencias, funciones y expresiones con precedencia.

Validación: detectar errores léxicos, sintácticos y semánticos.

Símbolos: administrar variables, funciones y ámbitos anidados.

Traducción: generar código de tres direcciones con temporales y etiquetas.

Optimización: aplicar transformaciones simples sobre el código intermedio.

Pruebas: verificar los componentes principales mediante JUnit y Maven.

2. DESARROLLO

2.1 Alcance del lenguaje

La gramática reconoce los tipos `int`, `float`, `double`, `char`, `string`, `bool` y `void`. Permite declaraciones e inicializaciones opcionales, asignaciones, arreglos unidimensionales, `cout`, expresiones aritméticas, relacionales y lógicas, `if-else`, `while`, `for`, `break`, `continue`, funciones, parámetros, llamadas y `return`.

El sistema no intenta ser compatible con todo C++. No incluye clases, punteros, referencias, plantillas ni generación de código máquina. Su finalidad es académica: mostrar un pipeline de compilación comprensible y extensible.

2.2 Arquitectura general

La clase `App` funciona como coordinadora. Carga el archivo, ejecuta cada fase y evita continuar cuando existen errores. ANTLR4 genera `MiLenguajeLexer`, `MiLenguajeParser` y las clases base del patrón `Visitor` a partir de `MiLenguaje.g4`. Las operaciones propias del proyecto se mantienen en clases separadas para no modificar el código generado.

```
Fuente -> Lexer -> Tokens -> Parser -> Árbol de parseo
        -> Visitor semántico -> Tabla de símbolos / errores
        -> CodigoVisitor -> Código de tres direcciones
        -> Optimizador -> Código optimizado / archivos de salida
```

2.3 Análisis léxico

El lexer transforma los caracteres en tokens. Las palabras reservadas se declaran antes que `ID` para impedir que términos como `int`, `while` o `return` se clasifiquen como identificadores. Los espacios y comentarios se descartan, mientras que cada token conserva su lexema, línea y columna.

`App` reemplaza los listeners predeterminados de ANTLR y muestra una tabla de tokens. La gramática posee una regla `OTRO` que captura caracteres no reconocidos; como estos tokens no forman parte de las reglas del parser, normalmente terminan reportados durante el análisis sintáctico.

2.4 Análisis sintáctico

El parser parte de la regla `programa`, compuesta por cero o más funciones o sentencias y el marcador `EOF`. Las expresiones utilizan alternativas etiquetadas para respetar la precedencia de operadores unarios, multiplicativos, aditivos, relacionales, de igualdad, `AND` y `OR`.

```
programa : elemento* EOF ;
elemento : funcion | sentencia ;
funcion  : tipo ID '(' parametros? ')' bloque ;
sentencia: declaracion | asignacion | if | while | for | return | bloque ;
```

Si la estructura es válida, ANTLR produce un árbol de parseo concreto. Al final de una compilación exitosa, `TreeViewer` abre una ventana `Swing` para visualizarlo.

2.5 Análisis semántico

`SemanticAnalyzerVisitor` recorre el árbol y utiliza `SymbolTable` para registrar variables, parámetros y funciones. Cada `Scope` mantiene sus símbolos y una referencia al ámbito

padre. La resolución comienza en el ámbito actual y continúa hacia el exterior, por lo que se admiten variables locales sin perder el acceso a símbolos globales.

La fase detecta declaraciones duplicadas, variables o funciones no declaradas, parámetros repetidos y usos de break, continue o return fuera de contexto. Las funciones se registran antes de recorrer los cuerpos, lo que permite realizar llamadas antes de su definición textual.

La implementación todavía no compara tipos de expresiones, asignaciones, argumentos y retornos. Tampoco valida la cantidad de argumentos contra la firma de una función. La infraestructura admite warnings, pero actualmente no existen reglas semánticas que los produzcan.

2.6 Tabla de símbolos

Symbol representa una variable o función mediante nombre, tipo, categoría y nivel de ámbito. SymbolTable ofrece operaciones de declaración, búsqueda y entrada o salida de ámbitos. La tabla que se imprime al finalizar muestra los símbolos conservados en el ámbito global; los símbolos locales se usan durante el recorrido, pero no se guardan en un historial para mostrarlos.

2.7 Generación de código intermedio

CodigoVisitor convierte las construcciones del árbol en instrucciones de tres direcciones. GeneradorCodigo crea temporales t0, t1, etc. para resultados parciales y etiquetas L0, L1, etc. para saltos. Las pilas de etiquetas de break y continue permiten traducir bucles anidados.

```
t0 = numeros[0] + numeros[1]
temp = t0
param temp
param 5
t1 = call sumar
if !t2 goto L0
```

Una limitación concreta es que visitDeclaracion emite la instrucción declare, pero no genera la asignación cuando la declaración posee inicializador. Por ejemplo, int x = 5 produce la declaración de x, pero no la instrucción x = 5.

2.8 Optimización

Las estrategias implementan la interfaz OptimizadorIntermedio. El proyecto incluye eliminación de código inalcanzable después de goto o return, propagación de constantes, simplificación de expresiones y eliminación de autoasignaciones. App utiliza por defecto OptimizadorCodigoMuerto.

La selección se realiza manualmente en App.java dejando activa una sola implementación. Por lo tanto, todavía no existe una cadena configurable que aplique varias optimizaciones en una misma compilación.

2.9 Salidas y diagnósticos

SalidaCompilador centraliza los mensajes importantes. Utiliza verde para [EXITO], amarillo para [WARNING] y rojo para [ERROR]. Esta organización mejora la lectura durante pruebas y exposiciones, aunque no modifica el resultado semántico del compilador.

El código intermedio se guarda con el sufijo `_codigo_intermedio.txt` y el resultado de la estrategia seleccionada con `_codigo_optimizado.txt`. La consola también presenta el número de tokens y un resumen de errores por fase.

2.10 Ejemplo representativo

El archivo `EJ_FINAL.txt` incluye variables globales, una función con parámetros, arreglos, operaciones aritméticas, una llamada con retorno y una estructura `if`. Este caso permite comprobar en una sola ejecución la mayor parte del recorrido implementado.

```
int sumar(int a, int b) {  
    int resultado;  
    resultado = a + b;  
    return resultado;  
}  
estado = sumar(temp, 5);
```

2.11 Pruebas y resultados

La suite se ejecutó con `mvn test` el 12 de junio de 2026. El resultado fue `BUILD SUCCESS` con 17 pruebas, 0 fallos, 0 errores y 0 omitidas. La cantidad se distribuye en `AppTest` (1), `SalidaCompiladorTest` (3), `SemanticAnalyzerVisitorTest` (6) y `SymbolTableTest` (7).

Las pruebas cubren la declaración y resolución de símbolos, duplicados, ámbitos anidados, funciones, parámetros, errores semánticos y colores ANSI. Aún faltan pruebas específicas para `CodigoVisitor` y para cada estrategia de optimización.

2.12 Limitaciones y mejoras futuras

AST: el sistema trabaja directamente sobre el árbol de parseo de ANTLR.

Tipos: no existe inferencia ni verificación completa de compatibilidad.

Funciones: no se validan cantidad y tipo de argumentos ni retorno.

Arreglos: no se controla el rango ni el tipo del índice.

Inicialización: el generador omite la asignación inicial de declaraciones.

Ámbitos: la impresión final solo conserva los símbolos globales.

Optimización: solo puede seleccionarse una estrategia por ejecución.

Backend: no se genera código máquina, bytecode ni ensamblador.

3. CONCLUSIONES

El proyecto integra correctamente las etapas principales de un compilador educativo. ANTLR4 redujo la complejidad de implementar el lexer y el parser, mientras que el patrón Visitor permitió separar la semántica y la generación de código de las clases generadas automáticamente.

La solución procesa un subconjunto significativo de C++, administra ámbitos y produce una representación intermedia comprensible. La división en componentes facilita identificar errores y permite extender el sistema sin modificar todo el flujo.

Los objetivos principales fueron alcanzados y las 17 pruebas confirman el funcionamiento de los componentes evaluados. Para una versión más completa sería necesario incorporar un AST propio, un sistema de tipos, validación de firmas, pruebas de generación y un pipeline de optimización configurable.

4. BIBLIOGRAFÍA

Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools*. Pearson.

Parr, T. (2013). *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf.

ANTLR. Documentación oficial. <https://www.antlr.org/>

Oracle. Documentación de Java. <https://docs.oracle.com/en/java/>

Apache Maven Project. Documentación oficial. <https://maven.apache.org/>

5. FUENTES DE DATOS

La información técnica fue contrastada con el código fuente del proyecto: `MiLenguaje.g4`, `App.java`, `SemanticAnalyzerVisitor`, `SymbolTable`, `Scope`, `CodigoVisitor`, `GeneradorCodigo`, las clases de optimización y las pruebas JUnit.

Repositorio:

https://github.com/Ignagalvan/PROYECTO_FINAL_TC_CALZADA_DOUGLAS_GALVAN

Verificación local: `mvn test` ejecutado el 12 de junio de 2026. Resultado: 17 pruebas, 0 fallos, 0 errores y 0 omitidas.

6. GLOSARIO

ANTLR4: herramienta que genera analizadores a partir de una gramática.

Lexer: componente que agrupa los caracteres de entrada en tokens.

Parser: componente que valida la estructura definida por la gramática.

Token: unidad léxica como una palabra reservada, operador o literal.

Parse tree: árbol concreto construido según las reglas reconocidas.

AST: representación abstracta del programa, no implementada actualmente.

Visitor: patrón utilizado para recorrer el árbol y ejecutar una operación.

Scope: ámbito que contiene símbolos y referencia a un ámbito exterior.

Tabla de símbolos: estructura que registra variables y funciones.

Código de tres direcciones: representación intermedia con operaciones simples.

Temporal: variable auxiliar usada para resultados parciales.

Optimización: transformación que mejora el código sin alterar su resultado.